3
Strings

Text strings are a fundamental data structure of growing importance. Internet search engines such as Google search billions of documents almost instantaneously. The sequencing of the human genome has given us three billion characters of text describing all the proteins we are built from. In searching this string for interesting patterns, we are literally looking for the secret of life.

The stakes in solving the programming problems in this chapter are considerably lower than that. However, they provide insight into how characters and text strings are represented in modern computers, and the clever algorithms which search and manipulate this data. We refer the interested reader to [Gus97] for a more advanced discussion of string algorithms.

3.1 Character Codes

Character codes are mappings between numbers and the symbols which make up a particular alphabet. Computers are fundamentally designed to work with numerical data. All they know about a given alphabet is which symbol is assigned to each possible number. When changing the font in a print program, all that really changes are the image bit-maps associated with each character. When changing an operating system from English to Russian all that really changes is the mapping of symbols in the character code.

It is useful to understand something about character code designs when working with text strings. The *American Standard Code for Information Interchange* (ASCII) is a

0	NUL	1	SOH	2	STX	3	ETX	4	EOT	5	ENQ	6	ACK	7	BEL
8	BS	9	HT	10	NL	11	VT	12	NP	13	CR	14	SO	15	SI
16	DLE	17	DC1	18	DC2	19	DC3	20	DC4	21	NAK	22	SYN	23	ETB
24	CAN	25	EM	26	SUB	27	ESC	28	FS	29	GS	30	RS	31	US
32	SP	33	!	34	"	35	#	36	$	37	%	38	&	39	'
40	(	41	)	42	*	43	+	44	,	45	-	46	.	47	/
48	0	49	1	50	2	51	3	52	4	53	5	54	6	55	7
56	8	57	9	58	:	59	;	60	<	61	=	62	>	63	?
64	@	65	A	66	B	67	C	68	D	69	E	70	F	71	G
72	H	73	I	74	J	75	K	76	L	77	M	78	N	79	O
80	P	81	Q	82	R	83	S	84	T	85	U	86	V	87	W
88	X	89	Y	90	Z	91	[	92	/	93	]	94	^	95	_
96	`	97	a	98	b	99	c	100	d	101	e	102	f	103	g
104	h	105	i	106	j	107	k	108	l	109	m	110	n	111	o
112	p	113	q	114	r	115	s	116	t	117	u	118	v	119	w
120	x	121	y	122	z	123	{	124	—	125	}	126	~	127	DEL

Figure 3.1. The ASCII character code.

single-byte character code where $2^7 = 128$ characters are specified.[1] Bytes are eight-bit entities; so that means the highest-order bit is left as zero.

Consider the ASCII character code table presented in Figure 3.1, where the left entry in each pair is the decimal (base-ten) value of the specification, while the right entry is the associated symbol. These symbol assignments were not done at random. Several interesting properties of the design make programming tasks easier:

- All non-printable characters have either the first three bits as zero or all seven lowest bits as one. This makes it very easy to eliminate them before displaying junk, although somehow very few programs seem to do so.
- Both the upper- and lowercase letters and the numerical digits appear sequentially. Thus we can iterate through all the letters/digits simply by looping from the value of the first symbol (say, "`a`") to value of the last symbol (say, "`z`").
- Another consequence of this sequential placement is that we can convert a character (say, "`I`") to its rank in the collating sequence (eighth, if "`A`" is the zeroth character) simply by subtracting off the first symbol ("`A`").
- We can convert a character (say "`C`") from upper- to lowercase by adding the difference of the upper and lowercase starting character ("`C`"-"`A`"+"`a`"). Similarly, a character x is uppercase if and only if it lies between "`A`" and "`Z`".
- Given the character code, we can predict what will happen when naively sorting text files. Which of "`x`" or "`3`" or "`C`" appears first in alphabetical order? Sorting alphabetically means sorting by character code. Using a different collating sequence requires more complicated comparison functions, as will be discussed in Chapter 4.
- Non-printable character codes for new-line (10) and carriage return (13) are designed to delimit the end of text lines. Inconsistent use of these codes is one of the pains in moving text files between UNIX and Windows systems.

More modern international character code designs such as *Unicode* use two or even three bytes per symbol, and can represent virtually any symbol in every language on

[1] Be aware that there are literally dozens of variations on ASCII. Perhaps the most important is ISO Latin-1, which is a full 8-bit code that includes European accented characters.

earth. However, good old ASCII remains alive embedded in Unicode. Whenever the high-order bits are 0, the text gets interpreted as single-byte characters instead of two-byte entities. Thus we can still use the simpler, more memory-efficient encoding while opening the door to thousands of additional symbols.

All of this makes a big difference in manipulating text in different programming languages. Older languages, like Pascal, C, and C++, view the `char` type as virtually synonymous with 8-bit entities. Thus the character data type is the choice for manipulating raw files, even for those which are not intended to be printable. Java, on the other hand, was designed to support Unicode, so characters are 16-bit entities. The upper byte is all zeros when working with ASCII/ISO Latin 1 text. Be aware of this difference when you switch between programming languages.

3.2 Representing Strings

Strings are sequences of characters, where order clearly matters. It is important to be aware of how your favorite programming language represents strings, because there are several different possibilities:

- *Null-terminated Arrays* — C/C++ treats strings as arrays of characters. The string ends the instant it hits the null character "\0", i.e., zero ASCII. Failing to end your string explicitly with a null typically extends it by a bunch of unprintable characters. In defining a string, enough array must be allocated to hold the largest possible string (plus the null) unless you want a core dump. The advantage of this array representation is that all individual characters are accessible by index as array elements.

- *Array Plus Length* — Another scheme uses the first array location to store the length of the string, thus avoiding the need for any terminating null character. Presumably this is what Java implementations do internally, even though the user's view of strings is as objects with a set of operators and methods acting on them.

- *Linked Lists of Characters* — Text strings can be represented using linked lists, but this is typically avoided because of the high space-overhead associated with having a several-byte pointer for each single byte character. Still, such a representation might be useful if you were to insert or delete substrings frequently within the body of a string.

The underlying string representation can have a big impact on which operations are easily or efficiently supported. Compare each of these three data structures with respect to the following properties:

- Which uses the least amount of space? On what sized strings?

- Which constrains the content of the strings which can possibly be represented?

- Which allow constant-time access to the ith character?

- Which allow efficient checks that the ith character is in fact within the string, thus avoiding out-of-bounds errors?
- Which allow efficient deletion or insertion of new characters at the ith position?
- Which representation is used when users are limited to strings of length at most 255, e.g., file names in Windows?

3.3 Program Design Example: Corporate Renamings

Corporate name changes are occurring with ever greater frequency, as companies merge, buy each other out, try to hide from bad publicity, or even raise their stock price – remember when adding a .com to a company's name was the secret to success!

These changes make it difficult to figure out the current name of a company when reading old documents. Your company, Digiscam (formerly Algorist Technologies), has put you to work on a program which maintains a database of corporate name changes and does the appropriate substitutions to bring old documents up to date.

Your program should take as input a file with a given number of corporate name changes, followed by a given number of lines of text for you to correct. Only *exact* matches of the string should be replaced. There will be at most 100 corporate changes, and each line of text is at most 1,000 characters long. A sample input is —

```
4
"Anderson Consulting" to "Accenture"
"Enron" to "Dynegy"
"DEC" to "Compaq"
"TWA" to "American"
5
Anderson Accounting begat Anderson Consulting, which
offered advice to Enron before it DECLARED bankruptcy,
which made Anderson
Consulting quite happy it changed its name
in the first place!
```

Which should be transformed to —

```
Anderson Accounting begat Accenture, which
offered advice to Dynegy before it CompaqLARED bankruptcy,
which made Anderson
Consulting quite happy it changed its name
in the first place!
```

The specifications do not ask you to respect word delimiters (such as blank), so transforming `DECLARED` to `CompaqLARED` is indeed the right thing to do.

Solution starts below

What kind of string operations do we need to do to solve this problem? We must be able to read strings and store them, search strings for patterns, modify them, and finally print them.

Observe that the input file has been segmented into two parts. The first section, the dictionary of name changes, must be completely read and stored before starting to convert the text. To declare the relevant data structures:

```
#include <string.h>

#define MAXLEN          1001    /* longest possible string */
#define MAXCHANGES      101     /* maximum number of name changes */

typedef char string[MAXLEN];

string mergers[MAXCHANGES][2];  /* store before/after corporate names */
int nmergers;                   /* number of different name changes */
```

We represent the dictionary as a two-dimensional array of strings. We do not need to sort the keys in any particular order, since we will be scanning through each of them on each line of text.

Reading the list of company names is somewhat complicated by the fact that we must parse each input line to extract the stuff between quotes. The trick is ignoring text before the first quote, and collecting it until the second quote:

```
read_changes()
{
        int i;                  /* counter */

        scanf("%d\n",&nmergers);
        for (i=0; i<nmergers; i++) {
                read_quoted_string(&(mergers[i][0]));
                read_quoted_string(&(mergers[i][1]));
        }
}

read_quoted_string(char *s)
{
        int i=0;                /* counter */
        char c;                 /* latest character */

        while ((c=getchar()) != '\"') ;
        while ((c=getchar()) != '\"') {
                s[i] = c;
                i = i+1;
        }
        s[i] = '\0';
```

```
}
```

The more advanced operations we will need are presented in the following sections.

3.4 Searching for Patterns

The simplest algorithm to search for the presence of pattern string p in text t overlays the pattern string at every position in the text, and checks whether every pattern character matches the corresponding text character:

```
/*      Return the position of the first occurrence of the pattern p
        in the text t, and -1 if it does not occur.
*/

int findmatch(char *p, char *t)
{
        int i,j;                        /* counters */
        int plen, tlen;                 /* string lengths */

        plen = strlen(p);
        tlen = strlen(t);

        for (i=0; i<=(tlen-plen); i=i+1) {
                j=0;
                while ((j<plen) && (t[i+j]==p[j]))
                        j = j+1;
                if (j == plen) return(i);
        }

        return(-1);
}
```

Note that this routine only searches for exact pattern matches. If a letter is capitalized in the pattern but not in the text there is no match. More seriously, if a company name is split between lines (see the example input), no match will be detected. Such searches can be performed by changing the text/pattern comparison from `t[i+j]==p[j]` to something more interesting. The same technique can be used to allow for *wild card* characters, which match anything. A more general notion of approximate string matching is discussed in Section 11.

This naive algorithm can take as much as $O(|p||t|)$ time in the worst case. Can you construct an arbitrary-length example pattern and text where it actually takes this much time without ever matching the pattern? Usually the naive search will be much more efficient, since we advance in the text soon as we get a single mismatch. More complicated, linear-time search algorithms do exist: see [Gus97] for a complete

discussion. These algorithms are likely implemented in the string library associated with your programming language.

3.5 Manipulating Strings

Manipulating strings requires knowing exactly which string representation you or your language is using. Here, we assume that strings are represented as a sequence of single-byte characters in a null-terminated array, as supported by C.

Treating strings like arrays makes many operations reasonably easy:

- *Computing the Length of a String* — Scan through the characters of the string, adding one to the count each time until we hit the null character.
- *Copying a String* — Unless your programming language supports copying entire arrays in one swoop, you must explicitly loop through each character one at a time. Remember to allocate enough memory for the new copy, and don't forget the null!
- *Reversing a String* — The easiest way to do this is to copy the string from right to left into a second array. The right endpoint is found by computing the string length. Remember to terminate the new string with a null! String reversal can also be done in place by swapping characters if you are willing to destroy the original string.

As an example, we implement a routine to replace a substring at a given position with another string. We will need it for our corporate merger program. The tricky part is moving the other characters in our string to accommodate the new text. If the replacement substring is longer than the original, we must move the suffix out of the way so it isn't stepped on. If the replacement substring is shorter, we must move the suffix in to cover up the hole:

```
/*      Replace the substring of length xlen starting at position pos in
        string s with the contents of string y.
*/

replace_x_with_y(char *s, int pos, int xlen, char *y)
{
        int i;                          /* counter */
        int slen,ylen;                  /* lengths of relevant strings */

        slen = strlen(s);
        ylen = strlen(y);

        if (xlen >= ylen)
                for (i=(pos+xlen); i<=slen; i++) s[i+(ylen-xlen)] = s[i];
        else
```

```
                for (i=slen; i>=(pos+xlen); i--) s[i+(ylen-xlen)] = s[i];

        for (i=0; i<ylen; i++) s[pos+i] = y[i];
}
```

An alternative approach would have been to assemble the new string in a temporary buffer, and then overwrite all the characters of s with the contents of the buffer.

3.6 Completing the Merger

With all of these supporting routines in place, the rest of the program becomes fairly simple:

```
main()
{
        string s;               /* input string */
        char c;                 /* input character */
        int nlines;             /* number of lines in text */
        int i,j;                /* counters */
        int pos;                /* position of pattern in string */

        read_changes();
        scanf("%d\n",&nlines);
        for (i=1; i<=nlines; i=i+1) {         /* read text line */
                j=0;
                while ((c=getchar()) != '\n') {
                        s[j] = c;
                        j = j+1;
                }
                s[j] = '\0';

                for (j=0; j<nmergers; j=j+1)
                        while ((pos=findmatch(mergers[j][0],s)) != -1) {
                           replace_x_with_y(s, pos,
                                strlen(mergers[j][0]), mergers[j][1]);
                        }

                printf("%s\n",s);
        }
}
```

3.7 String Library Functions

Whether you work in C, C++, or Java, be aware of the support provided for characters and strings through libraries or classes. There is no reason to reinvent the wheel.

Strings are not a supported data type of standard Pascal, and so the details vary with the specific implementation.

C Library String Functions

C contains both character and string libraries. The C language *character* library `ctype.h` contains several simple tests and manipulations on character codes. As with all C predicates, true is defined as any non-zero quantity, and false as zero.

```
#include <ctype.h>      /* include the character library */

int isalpha(int c);     /* true if c is either upper or lower case */
int isupper(int c);     /* true if c is upper case */
int islower(int c);     /* true if c is lower case */
int isdigit(int c);     /* true if c is a numerical digit (0-9) */
int ispunct(int c);     /* true if c is a punctuation symbol */
int isxdigit(int c);    /* true if c is a hexadecimal digit (0-9,A-F) */
int isprint(int c);     /* true if c is any printable character */

int toupper(int c);     /* convert c to upper case -- no error checking */
int tolower(int c);     /* convert c to lower case -- no error checking */
```

Check the definition of each carefully before assuming it does exactly what you want.

The following functions appear in the C language *string* library `string.h`. The full library has more functions and options, so check it out.

```
#include <string.h>      /* include the string library */

char *strcat(char *dst, const char *src);     /* concatenation */
int strcmp(const char *s1, const char *s2);   /* is s1 == s2? */
char *strcpy(char *dst, const char *src);     /* copy src to dist */
size_t strlen(const char *s);                 /* length of string */
char *strstr(const char *s1, const char *s2); /* search for s2 in s1 */
char *strtok(char *s1, const char *s2);       /* iterate words in s1 */
```

C++ String Library Functions

In addition to supporting C-style strings, C++ has a string class which contains methods for these operations and more:

```
string::size()          /* string length */
string::empty()         /* is it empty */
```

```
string::c_str()         /* return a pointer to a C style string */

string::operator [](size_type i)        /* access the ith character */

string::append(s)       /* append to string */
string::erase(n,m)      /* delete a run of characters */
string::insert(size_type n, const string&s) /* insert string s at n */

string::find(s)
string::rfind(s)        /* search left or right for the given string */

string::first()
string::last()          /* get characters, also there are iterators */
```

Overloaded operators exist for concatenation and string comparison.

Java String Objects

Java strings are first-class objects deriving either from the `String` class or the `StringBuffer` class. The `String` class is for static strings which do not change, while `StringBuffer` is designed for dynamic strings. Recall that Java was designed to support Unicode, so its characters are 16-bit entities.

The `java.text` package contains more advanced operations on strings, including routines to parse dates and other structured text.

3.8 Problems

3.8.1 WERTYU

PC/UVa IDs: 110301/10082, **Popularity:** A, **Success rate:** high **Level:** 1



A common typing error is to place your hands on the keyboard one row to the right of the correct position. Then "`Q`" is typed as "`W`" and "`J`" is typed as "`K`" and so on. Your task is to decode a message typed in this manner.

Input

Input consists of several lines of text. Each line may contain digits, spaces, uppercase letters (except "`Q`", "`A`", "`Z`"), or punctuation shown above [except back-quote (`)]. Keys labeled with words [`Tab`, `BackSp`, `Control`, etc.] are not represented in the input.

Output

You are to replace each letter or punctuation symbol by the one immediately to its left on the QWERTY keyboard shown above. Spaces in the input should be echoed in the output.

Sample Input

```
O S, GOMR YPFSU/
```

Sample Output

```
I AM FINE TODAY.
```

3.8.2 Where's Waldorf?

PC/UVa IDs: 110302/10010, **Popularity:** B, **Success rate:** average **Level:** 2

Given an m by n grid of letters and a list of words, find the location in the grid at which the word can be found.

A word matches a straight, uninterrupted line of letters in the grid. A word can match the letters in the grid regardless of case (i.e., upper- and lowercase letters are to be treated as the same). The matching can be done in any of the eight horizontal, vertical, or diagonal directions through the grid.

Input

The input begins with a single positive integer on a line by itself indicating the number of cases, followed by a blank line. There is also a blank line between each two consecutive cases.

Each case begins with a pair of integers m followed by n on a single line, where $1 \leq m, n \leq 50$ in decimal notation. The next m lines contain n letters each, representing the grid of letters where the words must be found. The letters in the grid may be in upper- or lowercase. Following the grid of letters, another integer k appears on a line by itself ($1 \leq k \leq 20$). The next k lines of input contain the list of words to search for, one word per line. These words may contain upper- and lowercase letters only – no spaces, hyphens, or other non-alphabetic characters.

Output

For each word in each test case, output a pair of integers representing its location in the corresponding grid. The integers must be separated by a single space. The first integer is the line in the grid where the first letter of the given word can be found (1 represents the topmost line in the grid, and m represents the bottommost line). The second integer is the column in the grid where the first letter of the given word can be found (1 represents the leftmost column in the grid, and n represents the rightmost column in the grid). If a word can be found more than once in the grid, then output the location of the uppermost occurrence of the word (i.e., the occurrence which places the first letter of the word closest to the top of the grid). If two or more words are uppermost, output the leftmost of these occurrences. All words can be found at least once in the grid.

The output of two consecutive cases must be separated by a blank line.

Sample Input

```
1

8 11
abcDEFGhigg
hEbkWalDork
FtyAwaldORm
FtsimrLqsrc
byoArBeDeyv
Klcbqwikomk
strEBGadhrb
yUiqlxcnBjf
4
Waldorf
Bambi
Betty
Dagbert
```

Sample Output

```
2 5
2 3
1 2
7 8
```

3.8.3 Common Permutation

PC/UVa IDs: 110303/10252, **Popularity:** A, **Success rate:** average **Level:** 1

Given two strings a and b, print the longest string x of letters such that there is a permutation of x that is a subsequence of a and there is a permutation of x that is a subsequence of b.

Input

The input file contains several cases, each case consisting of two consecutive lines. This means that lines 1 and 2 are a test case, lines 3 and 4 are another test case, and so on. Each line contains one string of lowercase characters, with first line of a pair denoting a and the second denoting b. Each string consists of at most 1,000 characters.

Output

For each set of input, output a line containing x. If several x satisfy the criteria above, choose the first one in alphabetical order.

Sample Input

```
pretty
women
walking
down
the
street
```

Sample Output

```
e
nw
et
```

3.8.4 Crypt Kicker II

PC/UVa IDs: 110304/850, **Popularity:** A, **Success rate:** average **Level:** 2

A popular but insecure method of encrypting text is to permute the letters of the alphabet. That is, in the text, each letter of the alphabet is consistently replaced by some other letter. To ensure that the encryption is reversible, no two letters are replaced by the same letter.

A powerful method of cryptanalysis is the known plain text attack. In a known plain text attack, the cryptanalyst manages to have a known phrase or sentence encrypted by the enemy, and by observing the encrypted text then deduces the method of encoding.

Your task is to decrypt several encrypted lines of text, assuming that each line uses the same set of replacements, and that one of the lines of input is the encrypted form of the plain text `the quick brown fox jumps over the lazy dog`.

Input

The input begins with a single positive integer on a line by itself indicating the number of test cases, followed by a blank line. There will also be a blank line between each two consecutive cases.

Each case consists of several lines of input, encrypted as described above. The encrypted lines contain only lowercase letters and spaces and do not exceed 80 characters in length. There are at most 100 input lines.

Output

For each test case, decrypt each line and print it to standard output. If there is more than one possible decryption, any one will do. If decryption is impossible, output

```
No solution.
```

The output of each two consecutive cases must be separated by a blank line.

Sample Input

```
1

vtz ud xnm xugm itr pyy jttk gmv xt otgm xt xnm puk ti xnm fprxq
xnm ceuob lrtzv ita hegfd tsmr xnm ypwq ktj
frtjrpgguvj otvxmdxd prm iev prmvx xnmq
```

Sample Output

```
now is the time for all good men to come to the aid of the party
the quick brown fox jumps over the lazy dog
programming contests are fun arent they
```

3.8.5 Automated Judge Script

PC/UVa IDs: 110305/10188, **Popularity:** B, **Success rate:** average **Level:** 1

Human programming contest judges are known to be very picky. To eliminate the need for them, write an automated judge script to judge submitted solution runs.

Your program should take a file containing the correct output as well as the output of submitted program and answer either `Accepted`, `Presentation Error`, or `Wrong Answer`, defined as follows:

Accepted: You are to report "`Accepted`" if the team's output matches the standard solution exactly. *All* characters must match and must occur in the same order.

Presentation Error: Give a "`Presentation Error`" if all *numeric* characters match in the same order, but there is at least one non-matching non-numeric character. For example, "`15 0`" and "`150`" would receive "`Presentation Error`", whereas "`15 0`" and "`1 0`" would receive "`Wrong Answer`," described below.

Wrong Answer: If the team output cannot be classified as above, then you have no alternative but to score the program a '`Wrong Answer`'.

Input

The input will consist of an arbitrary number of input sets. Each input set begins with a line containing a positive integer $n < 100$, which describes the number of lines of the correct solution. The next n lines contain the correct solution. Then comes a positive integer $m < 100$, alone on its line, which describes the number of lines of the team's submitted output. The next m lines contain this output. The input is terminated by a value of $n = 0$, which should not be processed.

No line will have more than 100 characters.

Output

For each set, output one of the following:

`Run #`x`: Accepted`
`Run #`x`: Presentation Error`
`Run #`x`: Wrong Answer`

where x stands for the number of the input set (starting from 1).

Sample Input

```
2
The answer is: 10
The answer is: 5
2
The answer is: 10
```

```
The answer is: 5
2
The answer is: 10
The answer is: 5
2
The answer is: 10
The answer is: 15
2
The answer is: 10
The answer is:  5
2
The answer is: 10
The answer is: 5
3
Input Set #1: YES
Input Set #2: NO
Input Set #3: NO
3
Input Set #0: YES
Input Set #1: NO
Input Set #2: NO
1
1 0 1 0
1
1010
1
The judges are mean!
1
The judges are good!
0
```

Sample Output

```
Run #1: Accepted
Run #2: Wrong Answer
Run #3: Presentation Error
Run #4: Wrong Answer
Run #5: Presentation Error
Run #6: Presentation Error
```

3.8.6 File Fragmentation

PC/UVa IDs: 110306/10132, **Popularity:** C, **Success rate:** average **Level:** 2

Your friend, a biochemistry major, tripped while carrying a tray of computer files through the lab. All of the files fell to the ground and broke. Your friend picked up all the file fragments and called you to ask for help putting them back together again.

Fortunately, all of the files on the tray were identical, all of them broke into exactly two fragments, and all of the file fragments were found. Unfortunately, the files didn't all break in the same place, and the fragments were completely mixed up by their fall to the floor.

The original binary fragments have been translated into strings of ASCII `1`'s and `0`'s. Your job is to write a program that determines the bit pattern the files contained.

Input

The input begins with a single positive integer on its own line indicating the number of test cases, followed by a blank line. There will also be a blank line between each two consecutive cases.

Each case will consist of a sequence of "file fragments," one per line, terminated by the end-of-file marker or a blank line. Each fragment consists of a string of ASCII `1`'s and `0`'s.

Output

For each test case, the output is a single line of ASCII `1`'s and `0`'s giving the bit pattern of the original files. If there are $2N$ fragments in the input, it should be possible to concatenate these fragments together in pairs to make N copies of the output string. If there is no unique solution, any of the possible solutions may be output.

Your friend is certain that there were no more than 144 files on the tray, and that the files were all less than 256 bytes in size.

The output from two consecutive test cases will be separated by a blank line.

Sample Input

```
1

011
0111
01110
111
0111
10111
```

Sample Output

```
01110111
```

3.8.7 Doublets

PC/UVa IDs: 110307/10150, **Popularity:** C, **Success rate:** average **Level:** 3

A *doublet* is a pair of words that differ in exactly one letter; for example, "booster" and "rooster" or "rooster" and "roaster" or "roaster" and "roasted".

You are given a dictionary of up to 25,143 lowercase words, not exceeding 16 letters each. You are then given a number of pairs of words. For each pair of words, find the shortest sequence of words that begins with the first word and ends with the second, such that each pair of adjacent words is a doublet. For example, if you were given the input pair "booster" and "roasted", a possible solution would be ("booster," "rooster," "roaster," "roasted"), provided that these words are all in the dictionary.

Input

The input file contains the dictionary followed by a number of word pairs. The dictionary consists of a number of words, one per line, and is terminated by an empty line. The pairs of input words follow; each pair of words occurs on a line separated by a space.

Output

For each input pair, print a set of lines starting with the first word and ending with the last. Each pair of adjacent lines must be a doublet.

If there are several minimal solutions, any one will do. If there is no solution, print a line: "`No solution.`" Leave a blank line between cases.

Sample Input

```
booster
rooster
roaster
coasted
roasted
coastal
postal

booster roasted
coastal postal
```

Sample Output

```
booster
rooster
roaster
roasted

No solution.
```

3.8.8 Fmt

PC/UVa IDs: 110308/848, **Popularity:** C, **Success rate:** low **Level:** 2

The UNIX program *fmt* reads lines of text, combining and breaking them so as to create an output file with lines as close to 72 characters long as possible without exceeding this limit. The rules for combining and breaking lines are as follows:

- A new line may be started anywhere there is a space in the input. When a new line is started, blanks at the end of the previous line and at the beginning of the new line are eliminated.
- A line break in the input may be eliminated in the output unless (1) it is at the end of a blank or empty line, or (2) it is followed by a space or another line break. When a line break is eliminated, it is replaced by a space.
- Spaces must be removed from the end of each output line.
- Any input word containing more than 72 characters must appear on an output line by itself.

You may assume that the input text does not contain any tabbing characters.

Sample Input

```
   Unix fmt

The unix fmt program reads lines of text, combining
and breaking lines so as to create an
output file with lines as close to without exceeding
72 characters long as possible.  The rules for combining and breaking
lines are as follows.

   1.  A new line may be started anywhere there is a space in the input.
If a new line is started, there will be no trailing blanks at the
end of the previous line or at the beginning of the new line.

   2.  A line break in the input may be eliminated in the output, provided
it is not followed by a space or another line break.  If a line
break is eliminated, it is replaced by a space.
```

Sample Output

```
   Unix fmt

The unix fmt program reads lines of text, combining and breaking lines
so as to create an output file with lines as close to without exceeding
72 characters long as possible.  The rules for combining and breaking
```

lines are as follows.

1. A new line may be started anywhere there is a space in the input. If a new line is started, there will be no trailing blanks at the end of the previous line or at the beginning of the new line.

2. A line break in the input may be eliminated in the output, provided it is not followed by a space or another line break. If a line break is eliminated, it is replaced by a space.

3.9 Hints

3.1 Should you use hard-coded logic to perform the character replacement, or would a table-driven strategy of initialized arrays be easier?

3.2 Can you write a single comparison routine with arguments which can handle comparison in all eight directions when called with the right arguments? Does it pay to specify directions as pairs of integers (δ_x, δ_y) instead of by name?

3.3 Can you rearrange the letters of each word so that the common permutation becomes more apparent?

3.5 What is the easiest way to compare just the numeric characters, as required for identifying presentation errors?

3.6 Can you easily figure out which pairs of fragments go together, if not their order?

3.7 Can we model this problem as a path problem in graphs? It might pay to look ahead to Chapter 9 where we present graph data structures and traversal algorithms.

3.10 Notes

3.4 Although it has a history dating back thousands of years, cryptography has been revolutionized by computational advances and new algorithms. Read Schneier's [Sch94] and/or Stinson's [Sti02] books to learn more about this fascinating area.

3.8 The gold standard among text-formatting programs is *Latex*, the system we used to typeset this book. It is built on top of *TeX*, developed by master computer scientist Don Knuth. He is the author of the famous *Art of Computer Programming* books [Knu73a, Knu81, Knu73b], which are still fascinating and unsurpassed more than 30 years after their original publication.